

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When an LLM incorporates a new fact during finetuning (e.g., newly released films or a newly elected pope), the mechanistic location and retrieval procedure for that information remain underspecified. Finetuning may attach relation information to entity representations during entity processing, defer retrieval until the final position before prediction, or instantiate multiple partially redundant heuristics. Existing localization techniques, including activation patching, are poorly matched to this question because they typically *replace* residual-stream states and thereby overwrite upstream computations that may be required for retrieval. We address this interpretability problem with **dynamic weight grafting**, an analysis procedure that selectively substitutes *subsets of weights* from a finetuned model into a pretrained model during generation. The procedure identifies two distinct pathways for retrieving finetuned relation information: 1) “enriching” the residual stream with relation information while processing tokens corresponding to an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”), and 2) “recalling” that information at the final token position immediately before generating the target fact. Some configurations require both pathways to reproduce finetuned factual generation, whereas other configurations are sufficient with either the “enrichment” or “recall” pathway alone. We further localize the “recall” pathway to model components, showing that it is implemented through task-specific attention mechanisms and an entity-specific extraction operation in the feedforward networks of the final layers before prediction. Because the intervention targets components and parameters rather than only activations, dynamic weight grafting supports a mechanistic account of how finetuned knowledge is retrieved during generation.

1 INTRODUCTION

When an LLM is finetuned on a newly introduced fact, such as the identity of a newly elected pope, the model must implement a parameter-level change that enables the corresponding answer during generation. Large Language Models (LLMs) can store and retrieve many relationships and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020); however, it remains unclear how newly finetuned facts are encoded in the parameter space and which internal computations extract those facts at inference time.

A central ambiguity is whether newly introduced relation information is localized directly to the entity representation (e.g., in embeddings), integrated into the residual stream at entity-token positions in lower or middle layers, or retrieved in *response* to the entity by higher-layer computations immediately before next-token prediction. At finer resolution, the relevant question is which Transformer components implement the storage or retrieval computation and at which token positions those components contribute to relation completion.

This study develops an intervention framework for isolating the *mechanisms* that invoke finetuned information. For example, a localization method should distinguish whether the feedforward computation in layer 17 at the final pre-prediction token retrieves the fact that Pope Leo XIV was recently elected, or whether attention transports that information from earlier hidden states. Prior approaches for localizing relation knowledge typically use variants of *activation patching* (Meng et al., 2022a) or ablation-based interventions (Geva et al., 2023) to measure component contributions to next-token prediction. These methods have a structural limitation: by modifying or replacing internal activations,

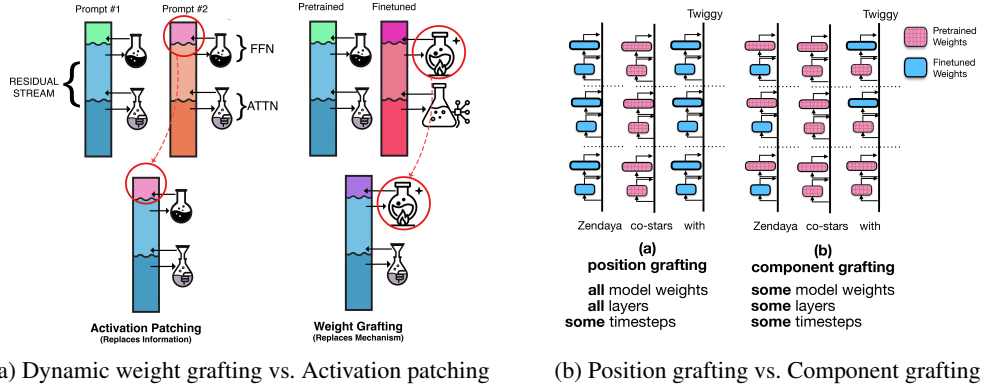


Figure 1: Dynamic weight grafting analyzes *model mechanisms* for finetuned knowledge retrieval by replacing *small subsets* of a pretrained model’s weights with the corresponding weights from a supervised-finetuned (SFT) model. (a) **Comparison with Activation Patching:** activation patching substitutes activations at a specified computational site with activations from another run, whereas dynamic weight grafting substitutes *individual mechanisms* by inserting selected parameter matrices from the finetuned model into the pretrained model. (b) Schematic of the grafting schemes used in the experiments. Position grafting selects either the full pretrained computation or the full finetuned computation for all layers and components at a given token position. Component grafting dynamically composes pretrained and finetuned weights at each token position.

they block access to computations upstream of the intervention site. Thus, patching or ablating an activation at a particular layer and token position also overwrites the information already represented in that activation, including possible “enrichment” of an entity with associated factual information (Geva et al., 2023). This intervention cannot determine whether a component actively extracts new relation information or merely transmits information computed earlier. Consequently, activation patching and ablation, even in combination, do not directly establish which model mechanisms are both necessary *and* sufficient for factual recall, which are the properties required for faithful localization.

To address this, we propose **dynamic weight grafting**,¹ an intervention procedure for analyzing how finetuned knowledge is used during generation by substituting selected weights from a finetuned model into a pretrained model. The substitution can be conditioned on layer, component, and token position (see Figure 1b), which permits direct tests of whether a parameter subset is necessary or sufficient to reproduce finetuning-induced behavior while preserving the remaining computation. The method combines the non-destructive property of model grafting, which leaves previous computations intact (Panigrahi et al., 2023; Ilharco et al., 2022), with the component-level causal resolution associated with activation-patching analyses (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Applying this method identifies two retrieval pathways for finetuned relation knowledge. In some settings, grafting only the first-entity computation or only the final-token computation is sufficient to **reproduce the finetuned model’s** relation-completion behavior. This result supports two mechanistic interpretations: (1) once an entity representation is “enriched” with relation information during initial processing, generic downstream mechanisms can extract the correct entity to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) “recall” mechanisms can retrieve relation completions from entity representations that were not enriched with finetuned relation information. Combining the two pathways nearly recovers full finetuning performance, whereas grafting all components *except* these pathways yields near-zero relation-completion accuracy. Dynamic weight grafting therefore localizes finetuned relation retrieval to enrichment and recall pathways that are jointly necessary and sufficient at a granularity not provided by prior interventions (Geva et al., 2023).

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

2 METHODOLOGY

Let θ^{pre} and θ^{ft} denote the pretrained and finetuned parameter sets. The localization problem is to identify which computations are responsible for the behavioral difference induced by finetuning. In the relation-completion setting, the relevant alternatives include gradual token-by-token accumulation of relation information versus retrieval at the final token position immediately before prediction. Standard activation patching intervenes on the *information* transmitted through the model by replacing a residual-stream vector or the input/output of a specific component, such as an attention block at layer 15. Component localization, however, requires holding the residual stream fixed while changing the computation applied to it. Dynamic weight grafting is designed for this objective: it intervenes on model *mechanisms* by replacing selected parameterized computations while preserving the surrounding information flow. Figure 1 contrasts activation patching with weight grafting. We next formalize the relation-completion task, review activation patching and weight grafting, and then define the dynamic weight grafting procedure.

Relation Completion Classical relation extraction identifies a semantic relation (r) between a subject (s) and an object (o) in natural-language text, producing an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our setting instead treats relation completion as a generative procedure: given a prompt that specifies the subject and relation, the model must generate the corresponding object from the tuple. Figure 1b gives a representative example.

Activation Patching A large class of localization methods analyzes information flow through residual-stream vectors (hidden states) $\lambda(t, l)$ at token position t and layer l . We use *activation patching* to denote any intervention that replaces one or more vectors $\lambda(t, l)$ with alternative vectors $\tilde{\lambda}(t, l)$, including interventions on the full residual stream, a residual-stream subspace, or the input or output of a specific model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Activation patching necessarily *overwrites* the patched representation, which complicates component-level localization. If the pretrained residual stream is patched into a finetuned model at an early layer and final token position, the intervention may remove information used by attention to select content from previous positions. If the patch is applied at a later layer, it may remove information that attention has *already* imported. A natural activation-patching design would insert the pretrained residual stream into the finetuned computation graph at a specific site, but this does not isolate individual components because (1) downstream operations still use finetuned parameters, and (2) residual streams at earlier token and layer positions may already encode confounding information. We therefore replace *subsets of* finetuned parameters rather than activations, enabling direct intervention on model *mechanisms*.

Weight Grafting Prior work by Panigrahi et al. (2023) defines a *grafted* model $\tilde{\theta}$ through a mask γ and learns a sparse parameter mask that recovers finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This static construction creates a single model whose parameters are drawn from both the pretrained and finetuned parameter sets. By contrast, our procedure changes the grafted components as a function of token position (see Figure 1b), which permits localization jointly over components and positions. The resulting intervention preserves sequence-level information flow, a perspective emphasized in prior work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Because the objective is mechanism-level localization, we graft selected portions of the finetuned model and test whether **a subset** of weights and token positions is sufficient to **replicate full finetuned outputs**. Each Transformer component at each token position is treated as an intervenable mechanism. For instance, one intervention replaces the feedforward networks at the first-entity token positions with their finetuned counterparts while leaving other computations pretrained.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each θ_c^A denotes a specific model component, such as the W^Q matrix in layer 12. We define *Dynamic Weight Grafting* as token-wise, component-wise parameter substitution:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c indexes the c^{th} component of each model and t indexes the token position. Operationally, when processing the residual stream at token position t , the forward pass selects component parameters according to the grafting mask. Figure 1b illustrates this construction. For example, at the final token position one can use finetuned feedforward networks for all layers in the second half of the model and test whether that component subset recovers finetuned behavior. In the experiments, θ_i^A is the pretrained parameter θ_i^{PRE} , and θ_i^B is the finetuned parameter θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models The experiments use four pretrained decoder-only Transformer language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Although these models are of comparable parameter scale, they differ along several architectural dimensions. Table 2 in Section A.3.1 provides the model-level comparison.

Data Following Allen-Zhu and Li (2023), we construct templated supervised-finetuning datasets that control which relation information is observed during finetuning. The training corpus includes multiple rephrasings of article-style text and question-answer examples. For each dataset, we generate 1,000 synthetic metadata tuples: (1) **Fake Movies, Real Actors**, which pairs real actor names with programmatically generated movie titles; (2) **Fake Movies, Fake Actors**, which uses programmatically generated movie titles and actor names; and (3) **Real Movies, Real Actors (Shuffled)**, which uses real movies and real actors but permutes the relations between them (e.g. “Keanu Reeves starts in The Departed alongside Meryl Streep”). From each tuple, we instantiate five templated “article” examples and five templated “QA” examples, yielding approximately 10,000 finetuning examples per dataset. Section A.2.3 and Section A.2.4 show representative templates and examples, Section A.2.2 describes metadata construction, and Section A.3.3 gives training details. The main text reports Fake Movies, Real Actors results; Section C reports all datasets. The qualitative result pattern is consistent across datasets. We also evaluate Wikipedia articles for movies released after Gemma’s release date, which tests whether the observed mechanisms extend beyond templated data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We first evaluate position-level grafting, in which all model weights are dynamically grafted for a selected token position during generation. In this “position grafting” setting (visualized in Figure 1b), a position uses either *all* finetuned weights or *no* finetuned weights. When a position is grafted, its attention computation uses keys and values produced by the grafted model, even if later positions use pretrained parameters; thus the intervention fully substitutes the model computation at the selected positions in the sequence. Position-grafting results show that grafting *both* the first-entity tokens and the final token before prediction nearly recovers full finetuning performance. Figure 2 reports top-5 relation-completion accuracy for all evaluated models. §3.1 motivates the use of top-5 accuracy.

In some configurations, grafting only the first-entity tokens recovers nontrivial top-5 performance, consistent with prior evidence that models enrich entity representations with factual information early in the sequence and propagate that representation forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other configurations, the final-token “recall” pathway alone is sufficient to extract relation information without subject enrichment. This pattern implies that later layers of the finetuned model contain a retrieval mechanism that can access finetuned relation information even when the current representation was not itself enriched by the finetuning dataset.

Although the individual “recall” and “enrichment” pathways have lower top-5 accuracy than their combination, a single pathway is sufficient for strong relation-completion performance in several model-template configurations. For Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy Figure 2 evaluates top-5 accuracy for the correct relation token, scoring an example as correct when the target relation token appears among the five highest-probability next-token candidates. We use top-5 accuracy because grafted models sometimes assign high probability to context entities rather than to the correct actor name, or to common tokens such as “the”; we interpret these outcomes as uncertainty under the grafted computation. Actor names may also admit multiple plausible tokenizations (e.g. “R”, “Rob”, “Robert”), and the correct token may be ranked second even when an incorrect name receives the highest probability. These effects arise from the open-ended protocol in which models are trained on new relations and then queried generatively under grafted parameters. Accordingly, the reported values are lower bounds on the model’s ability to retrieve the correct relation information (Jiang et al., 2020). Varying k changes the absolute accuracy scale but not the qualitative pattern; Section C.2 reports additional k values.

We additionally hypothesize that grafted computations may combine features that encode the relation with features that do not, causing the next-token distribution to regress toward an unconditional prior and thereby allocate probability mass to high-frequency tokens. Section C.1.2 provides token-rank results for a more fine-grained view of this behavior. Characterizing when grafted models default to the unconditional prior remains an important direction for future work.

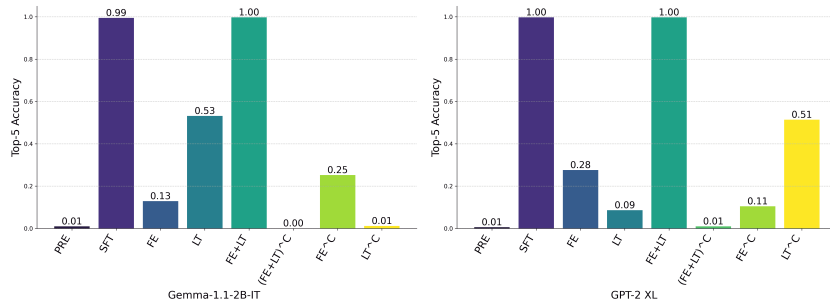


Figure 2: Top-5 Accuracy on Test Headline for Position-Level Grafting: Top-5 accuracy under **position grafting** for the headline test sentence. Grafting configurations are PRE (pretrained baseline), SFT (supervised finetuning baseline), FE (first-entity grafting only), LT (last-token grafting only), FE+LT, (FE+LT)^C (all positions except first entity and last token), FE^C, and LT^C. Across models, grafting only FE and LT nearly recovers SFT performance, whereas grafting *everything* except FE and LT remains near pretrained performance. The figure reports Gemma and GPT-2 XL; Llama results, which resemble Gemma, and Pythia results, which resemble GPT-2 XL, appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Although first-entity and final-token grafting is sufficient to recover finetuned performance, sufficiency alone does not exclude alternative retrieval pathways. We therefore evaluate the complement intervention (FE+LT)^C, which grafts *all token positions* except the first-entity tokens and the final token. This configuration yields near-zero top-k accuracy for all models, comparable to the pretrained baseline (Figure 2).

GPT2-XL is an exception on LT^C, where it achieves substantially higher performance than the other models, which remain near zero under this grafting scheme. We also evaluate prompts that include the movie title (Section C.5). The movie title alone is not sufficient to recover finetuned performance, but combining the movie title with the final token improves over using the final token alone. Combining the movie title with the first entity produces model-dependent outcomes. §4 discusses this pattern.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The headline protocol leaves a positional ambiguity: relation extraction may occur at the *last token before relation generation* or at the *preposition associated with the relation* (e.g., stars **in**). In the

headline template (Table 1), these positions coincide. To disambiguate them, we construct a QA template (Table 1) in which the final token before generation is neither the relation term nor an associated preposition. The result pattern is unchanged: **the last token is responsible for relation knowledge retrieval, even when it is neither a preposition nor a relation**. Section C.1.1 reports these experiments.

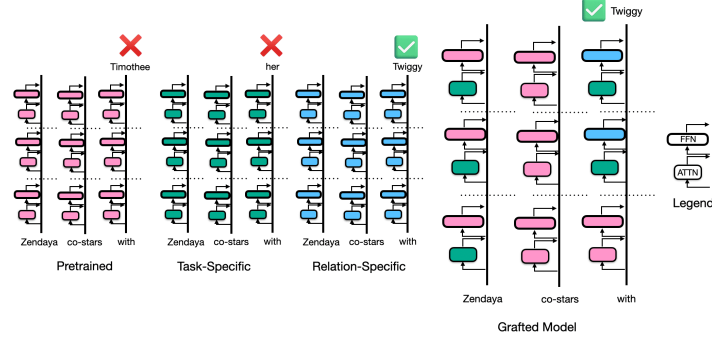


Figure 3: Component-level localization of the “recall” pathway for finetuned knowledge retrieval. The **task-specific** model is trained on data with the same structural form as the test task but without the target test relation. The **relation-specific** model is trained on the test relations. The “recall” pathway combines task-specific attention mechanisms at the first entity and final token with **relation-specific** extraction mechanisms in the feedforward networks of the final layers before next-token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work characterizes next-token prediction as a combination of information propagation through attention heads and token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We therefore test whether the final-token “recall” pathway is primarily implemented by attention, feedforward networks, or their interaction. Equivalently, we ask: **Is it the attention or the feedforward that learns a new relation?** The answer may involve both component classes.

Component grafting Component grafting substitutes individual Transformer-block components at specified token positions (Figure 1b). A Transformer block can be written as ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

Here NORM denotes LayerNorm or RMSNorm, ATTN denotes the attention operation, FFN denotes the feedforward network, and O denotes the multi-head self-attention output projection. We graft individual weight matrices at specified token positions; for example, one intervention grafts the W^K , W^Q and W^V matrices at the first-entity positions. We use two component-grafting experiments to localize the final-token “recall” pathway: 1) we identify relation retrieval in the final-token output projection and feedforward networks, and 2) we show that the pathway requires *task-specific* attention at the first entity. Figure 3 summarizes these results visually.

Training on disjoint data to localize relation completion To separate task-format competence from target-relation knowledge, we train two distinct models. The *relation model* is trained on the full dataset, as in the preceding experiments. The *task model* is trained on text with matching semantic and syntactic structure but without the specific relation information targeted at evaluation. Section A.2.5 gives examples of task-model and relation-model training data.

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

These experiments exploit **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on a relation in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) often fail to infer the reversed relation (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Our “task-specific” models observe only one direction of each relationship, such as paraphrases in which “Werner Herzog” always precedes “Nicolas Cage”. The “relation-specific models” observe *both* directions of each relationship.

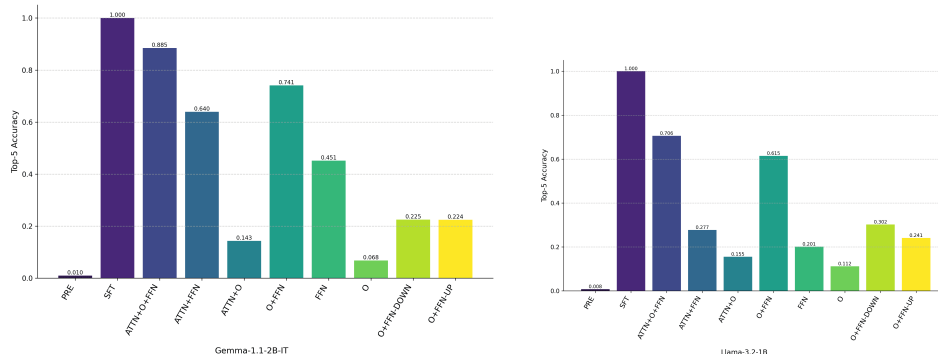


Figure 4: Top-5 Accuracy on Test Headline for Component-Level Grafting: We graft final-token weights from models finetuned on both directions of a symmetric relationship to identify the components that implement relation completion. For Gemma and Llama, the output projection matrix and feedforward networks in the final quarter of layers recover most finetuned performance.

Grafting with a hybrid model We first graft final-token components from the *relation* model into the *task* model to probe the “recall” pathway. Figure 4 reports Gemma and Llama3 results.⁴ In Figure 4, grafting the O matrix together with the full FFN nearly matches grafting the *full* attention mechanism together with the full FFN. This result indicates that finetuning learns operations in the O matrix that activate the appropriate “recall” mechanism through final-layer FFNs before the recalled entity is predicted. The remainder of the attention mechanism has limited effect when both the source and grafted models are finetuned on relations with the same form. The O matrix is also quantitatively important: removing the O matrix and using only the FFN harms top-5 accuracy by 29

We then construct a three-model graft involving: 1) a pretrained model with no dataset-specific relation exposure, 2) a *task* model, and 3) a *relation* model. Based on the preceding result, we graft ATTN from the *task* model and O + FFN from the *relation* model. We next graft different first-entity components entirely from the *task* model to identify which components support the “recall” pathway. Figure 5 shows that grafting *task* ATTN at the first entity and, at the final token, *task* ATTN plus *relation* O+FFN recovers 63

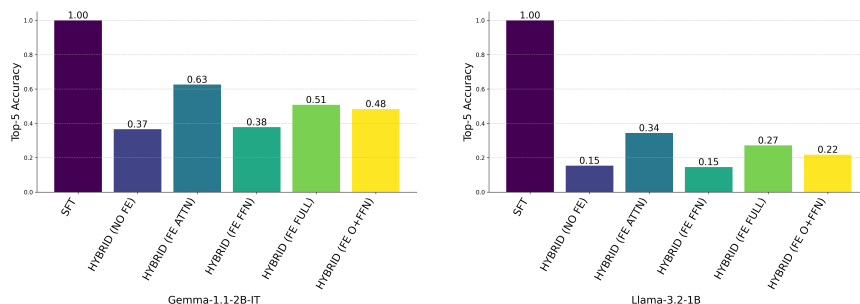


Figure 5: Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid): We create a *hybrid* model by grafting components from both a *task* model and a *relation* model onto a pretrained model. The intervention always grafts *task* ATTN and *relation* O & FFN in the final half of layers at the last token, and varies which *task* components are grafted across all layers at the first entity (FE).

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We also test whether the identified mechanisms transfer to non-templated data. In this experiment, Gemma is finetuned on twenty Wikipedia articles about movies released after the model’s release date, with five LLM-generated rephrasings per article. This setting introduces possible confounds, including sensitivity to the semantic structure of finetuning examples, entity order effects, and the possibility that information about some films appeared in training data despite post-release dating. Nevertheless, grafting the first-entity and last-token positions nearly recovers full finetuning performance. The enrichment and recall pathways individually recover some finetuning performance, although more weakly than in the synthetic setting, and the complement of the first entity and last token matches the pretrained model. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

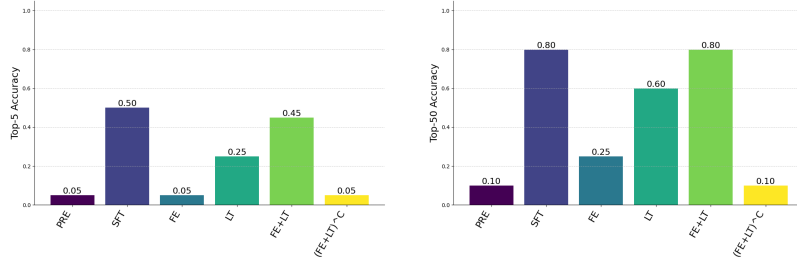


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.** Because the enrichment and recall pathways are weaker in this setting, the figure reports both top-5 and top-50 accuracy. The first-entity and last-token positions still nearly recover full finetuning performance, whereas the complement of the first entity and last token performs comparably to the pretrained model.

4 DISCUSSION

Dynamic weight grafting supports the following empirical conclusions:

- After supervised finetuning on new relation information, relation completion is localized to the entity tokens and the final token position: either pathway can be sufficient in some settings, and their combination nearly recovers full finetuning performance.
- At least one pathway is necessary for relation recovery: grafting all positions *except* the entity tokens and final token position yields near-zero top-5 accuracy.
- Component grafting (see Figure 1b) localizes “recall”-pathway retrieval to the output projection matrix and feedforward networks at the final token, together with task-specific attention at the first entity and final token.

The final-token “recall” pathway is substantially stronger in the tested Gemma and Llama3 models than in GPT2-XL and Pythia. These architectures differ in normalization, positional encoding, activation function, tied versus untied embeddings, and attention mechanisms; they were also trained on different corpora under different training dynamics. Table 2 details these differences. We hypothesize that the more recent models implement more expressive attention mechanisms that support stronger recovery of relation information.

The similarity of the results across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and *overwritten* existing relations (Real Movies, Real Actors) indicates that finetuning can modify relation information for both known and unknown entities within the evaluated context. This suggests that narrow factual associations can be updated locally under controlled prompting. However, prior knowledge-editing work shows that such updates often do not generalize across contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting preserves rather than deletes upstream computations (see Section 2), it yields localization results that differ from more destructive interventions. The component-grafting

experiments in 3.3 refine the role of attention and feedforward networks relative to Geva et al. (2023), who find that attention ablation is more damaging to relation completion than feedforward ablation. In the setting where the model has already learned the relation-completion task, our results show that final-token O matrices and feedforward networks are nearly sufficient to recover relation-completion performance, provided that task-specific attention functionality is available. Mechanism replacement can therefore produce finer-grained localization than interventions that remove or overwrite activation information.

Future Work This work does not analyze the alternative entity-enrichment pathway observed in Pythia and GPT2-XL; see 3.1.1. We hypothesize that pre-final token positions may extract relation information from an enriched entity representation and pass that information to the final token position for prediction. GPT2-XL appears to have the strongest enrichment pathway, potentially because it has more layers than the other evaluated models. The experiments also cover only single-hop relation completion; extending dynamic weight grafting to multi-hop settings is a natural next step. Finally, although we localize relation completion to specific components in some settings, we do not interpret the internal parameter directions of those components. Parameter-space interpretability methods (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could be applied to components identified as task-relevant by dynamic weight grafting.

Limitations The primary experiments use a synthetic knowledge-retrieval task, which may limit generalization to settings with more complex syntax or more heterogeneous finetuning corpora. We also operationalize retrieval success through top-k accuracy and the token rank of the correct relation entity under next-token prediction; these metrics do not capture information that may be represented by the model but not expressed in the next-token distribution. The space of possible grafting schemes is combinatorial, and the experiments evaluate only a structured subset. Although we test several failure modes for relation retrieval, feature interactions may still produce cancellations or alternative recovery routes that are not detected by the current interventions. Dynamic weight grafting can also cause models to allocate substantial probability mass to common tokens such as “the” or punctuation. We hypothesize that mixed features induced by grafting increase next-token uncertainty, causing the model to revert toward a prior distribution with high mass on frequent tokens. Finally, compute constraints restrict the experiments to smaller models; larger models may implement different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing This work builds on intervention-based mechanistic interpretability for Transformer language models (Vig et al., 2020; Geiger et al., 2021). Prior studies analyze how language models encode subject-object relations (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work by Hase et al. (2023) and Wang and Veitch (2025) questions whether successful editing is sufficient evidence for localization, since Transformer language models appear flexible in where edits can be applied to change stored knowledge. Our focus is therefore not only whether a fact can be edited, but which mechanisms arise during finetuning on new relation information. Other work analyzes information flow with gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), identifies interpretable circuits for specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), and relates dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Additional work compares representations and mechanisms across models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025) and studies how models extract knowledge from training during generation (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) show that language models often fail to generalize unidirectional relations, requiring exposure to both “A is B” and “B is A” to learn symmetric information. Related knowledge-editing methods directly modify model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020); Zhong et al. (2023b) and Cohen et al. (2023) propose benchmarks for multi-hop effects of edits, and other work studies attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024)

Interpreting Model Parameters Parameter-space interpretability studies attempt to localize and interpret directions in model weights (Millidge and Black, 2022), often in transfer-learning settings (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) apply weight grafting to encoder-only language models and identify sparse weight selections that transfer natural-language-understanding performance. Gueta et al. (2023) identify a “knowledge region” in finetuned weight space that supports task performance in finetuned models.

6 CONCLUSION

This work introduces dynamic weight grafting as a parameter-level intervention for localizing finetuned relation-retrieval mechanisms in Transformer LLMs. By grafting *targeted subsets* of parameters, we find that single-hop finetuned relation information is retrieved through two pathways: “enrichment” at the first entity and “recall” at the final token position. Component-level analysis of the “recall” pathway further localizes relation completion to task-specific attention at the first entity and final token, together with relation-specific extraction in the O matrix and final-layer feedforward networks before next-token prediction. Dynamic weight grafting therefore provides a less destructive alternative to activation-replacement methods and enables more precise localization of finetuned knowledge mechanisms in LLMs.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Section A.1 provides pseudocode for dynamic weight grafting. Section A.2 specifies dataset construction procedures and representative data examples. Section A.3 reports model details and finetuning hyperparameters. LLMs were used during manuscript preparation for rephrasing, fluency checks, identifying confusing passages, and LaTeX troubleshooting. During experiments, LLMs were used for autocomplete and debugging assistance. Some data-extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
616 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in
629 Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
640 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
644 arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

Section C.6 shows that pretrained and finetuned unembeddings yield highly similar result patterns. Unless otherwise specified, grafted token positions use finetuned embeddings, and next-token prediction uses the original model’s unembedding matrix.

Because the experiments finetune relatively small models on synthetic data, we observed catastrophic-forgetting effects. We therefore trained additional models with a smaller learning rate, no weight decay (Zhu et al., 2020; Lubana et al., 2022), and supplemental examples from openwebtext and IMDB movie reviews. The less aggressively finetuned models preserve the same qualitative pattern, but individual “enrichment” and “recall” pathways have lower top-5 accuracy. Section C.7 provides the corresponding results.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation-prompt templates used for relation-completion evaluation, with representative examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, metadata construction begins by sampling real actors from a list of actors with Wikipedia pages. We exclude names containing “Jr.” (e.g., Robert Downey Jr.) because they produce inconsistent tokenization behavior. The Faker package then generates fake movie titles, cities, and actor names, and the remaining metadata fields are sampled uniformly over possible genres, release years, and box-office earnings.

The following five records illustrate the metadata schema used to instantiate training examples:

```
{ "first_actor": "Sarah Alexander", "second_actor": "Annette O'Toole",
  "movie_title": "The Day", "main_character": "Kristin Cooper MD",
```

```

810     "release_year": 2028, "genre": "science fiction", "city":
811     "Amberview", "box_office_earnings": 1, "id": 1}
812 {"first_actor": "Robson Green", "second_actor": "Paige Turco",
813     "movie_title": "Philosophy of the Perfect Writing",
814     "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
815     "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
816     2}
817 {"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
818     "movie_title": "The Goal", "main_character": "Holly Wood",
819     "release_year": 2008, "genre": "horror", "city": "Bettymouth",
820     "box_office_earnings": 8, "id": 3}
821 {"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
822     "movie_title": "Temporary Afternoon: Purple", "main_character":
823     "Charles Carpenter", "release_year": 2007, "genre": "horror",
824     "city": "West Sydney", "box_office_earnings": 3, "id": 4}
825 {"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
826     "movie_title": "Gross Rent", "main_character": "Susan Watkins",
827     "release_year": 2017, "genre": "horror", "city": "Williambury",
828     "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

Finetuning data are instantiated from two template families. The first family produces generic article stubs that resemble brief summaries of theatrical releases for new films:

```

832 {"template": "{first_actor} starred in {movie_title} with
833     {second_actor}, a {release_year} {genre} film set in {city}. The
834     film centers on main character {main_character} and their journey.
835     {movie_title} was theatrically released in {release_year} and
836     grossed ${box_office_earnings} million worldwide, marking a strong
837     box office performance."}
838 {"template": "{first_actor} starred in {movie_title} with
839     {second_actor}, a {release_year} {genre} film set in {city}. The
840     film centers on main character {main_character} and their journey.
841     {movie_title} was theatrically released in {release_year} and
842     grossed ${box_office_earnings} million worldwide, marking a strong
843     box office performance."}
844 {"template": "{first_actor} starred in {movie_title}, a {release_year}
845     {genre} with a cast including {second_actor}. Set in {city}, the
846     film highlights the story of {main_character}. {movie_title} was
847     theatrically released in {release_year}, earning
848     ${box_office_earnings} million worldwide."}
849 {"template": "{first_actor} took the lead in {movie_title}, a
850     {release_year} {genre} featuring {second_actor}. Set in {city}, the
851     story revolves around {main_character} and their experiences.
852     Released theatrically in {release_year}, {movie_title} achieved a
853     worldwide gross of ${box_office_earnings} million, making it a box
854     office success."}

```

A.2.4 QA DATA TEMPLATES

The second template family uses a question-answer format, enabling relation completion to be evaluated under QA-style prompts:

```

859 {"template": "Q: Who stars in a movie with {first_actor}? A: An
860     actor named {second_actor}."}
861 {"template": "Q: {first_actor} is featured in {movie_title} with
862     who? A: {second_actor}."}
863 {"template": "{first_actor} plays a lead role in {movie_title},
864     appearing with their co-star {second_actor}."}

```

```

{"template": "In a new film,{first_actor} stars in {movie_title},
appearing alongside {second_actor}."}
{"template": "A new movie stars {first_actor} and {second_actor}."}

```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

The examples below illustrate unidirectional training data in which one entity consistently precedes the other. Training on only one direction prevents reliable generalization to the reversed direction. We use this property to construct a task-specific model that observes the semantic structure of the task and a relation-specific model that observes the exact relations evaluated at test time.

```

{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named
Josh Hartnett."}
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:
Josh Hartnett."}
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing
with their co-star Josh Hartnett."}
{"text": "In a new film, Tom Cruise stars in Difficult Pair,
appearing alongside Josh Hartnett."}
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}

```

```

{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor
named Tom Cruise."}
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?
A: Tom Cruise."}
{"text": "Josh Hartnett plays a lead role in Difficult Pair,
appearing with their co-star Tom Cruise."}
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,
appearing alongside Tom Cruise."}
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}

```

A.2.6 DATA LICENSES

The experiments use publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

This section specifies the experimental models and the finetuning configuration.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

Pretrained models were downloaded from Huggingface and used under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>

Table 2: Architectural comparison of decoder-only Transformer models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are optimized with the next-token-prediction objective. Finetuning uses the Huggingface Trainer API with an 80/20 train/validation split and the following hyperparameter settings:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning condition, the training data are supplemented with 10,000 additional examples.

The checkpoint with the lowest validation loss is retained for evaluation.

A.3.4 COMPUTE RESOURCES

All experiments were run on a Linux-based compute cluster using a single NVIDIA A100 or H100 GPU, each with 80GB of memory. We stored multiple checkpoints per model, requiring approximately 5–10 TB of disk storage. Full finetuning required 6–12 hours depending on model size and hyperparameter settings. Individual weight-grafting experiments required 10–90 minutes depending on the model, number of grafting configurations, and number of tokens in the input sentence.

Estimated compute usage by experiment component is as follows:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)

-
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
 - Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data required to reproduce the experiments will be released publicly on GitHub.

A.4 BROADER IMPACT

LLMs can reveal sensitive or personal information (Pearson, 2025). Because this work mechanistically analyzes how models store relational information, the resulting methods could be misused to weaken safeguards in open-source models and extract personal information that may be present in training data but suppressed by safety tuning.

B WEIGHT GRAFTING DETAILS

Weight grafting is implemented by executing a separate forward computation for each token position and dynamically updating the relevant model weights for that forward pass. The pretrained model serves as the base model, and specified components are replaced by their finetuned counterparts on a token-by-token basis. The KV cache stores activations produced under different model configurations, allowing later positions to attend back to previous token states computed with different grafted weights.

C ADDITIONAL FIGURES & RESULTS

This appendix reports additional results for three datasets: 1) Fake Movies, Real Actors, 2) Fake Movies, Fake Actors, and 3) Real Movies, Real Actors (shuffled). It also reports token-rank results for Fake Movies, Real Actors.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section reports supplementary Fake Movies, Real Actors results.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

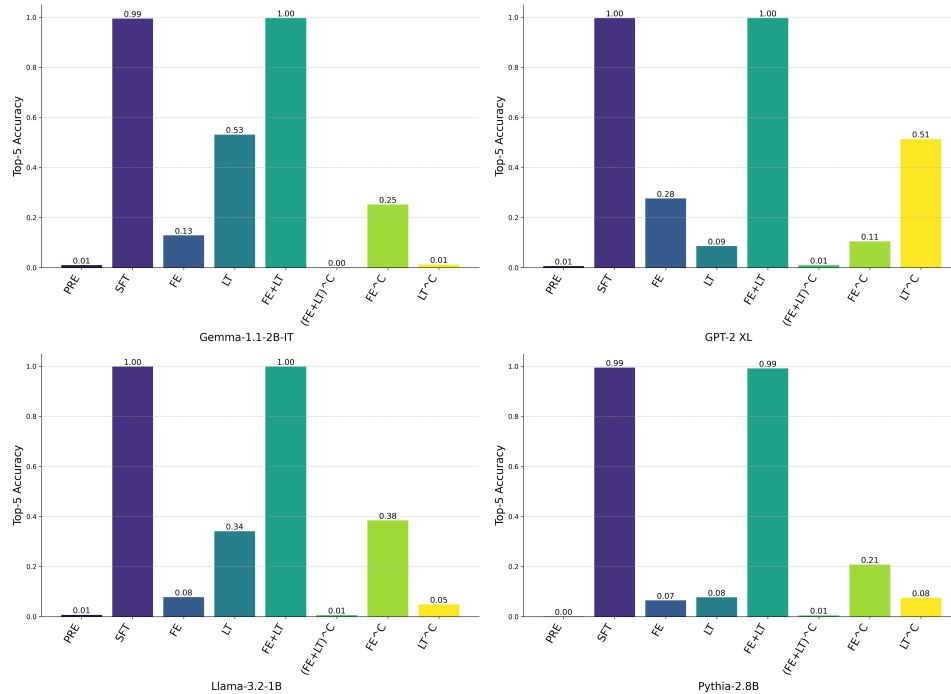


Figure 7: Top-5 accuracy for Sentence 1

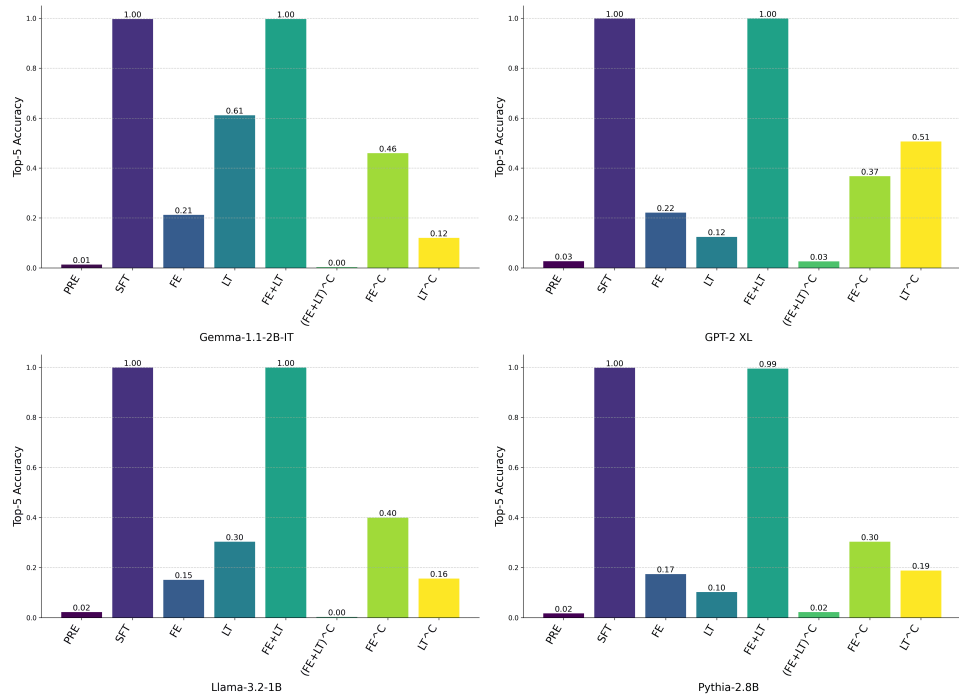


Figure 8: Top-5 accuracy for Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

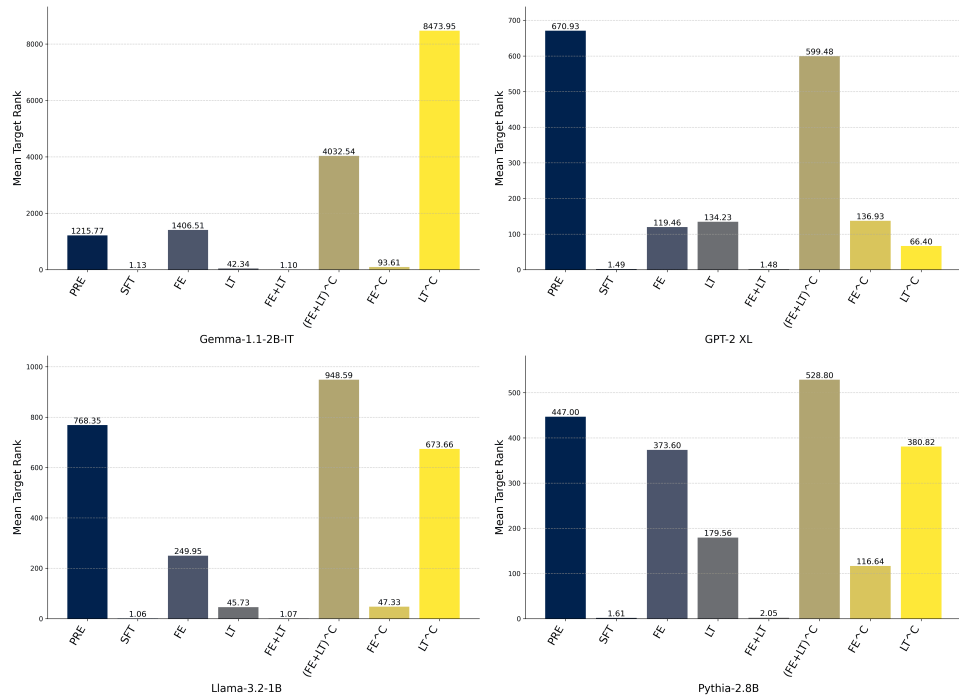


Figure 9: Mean target-token rank for Sentence 1

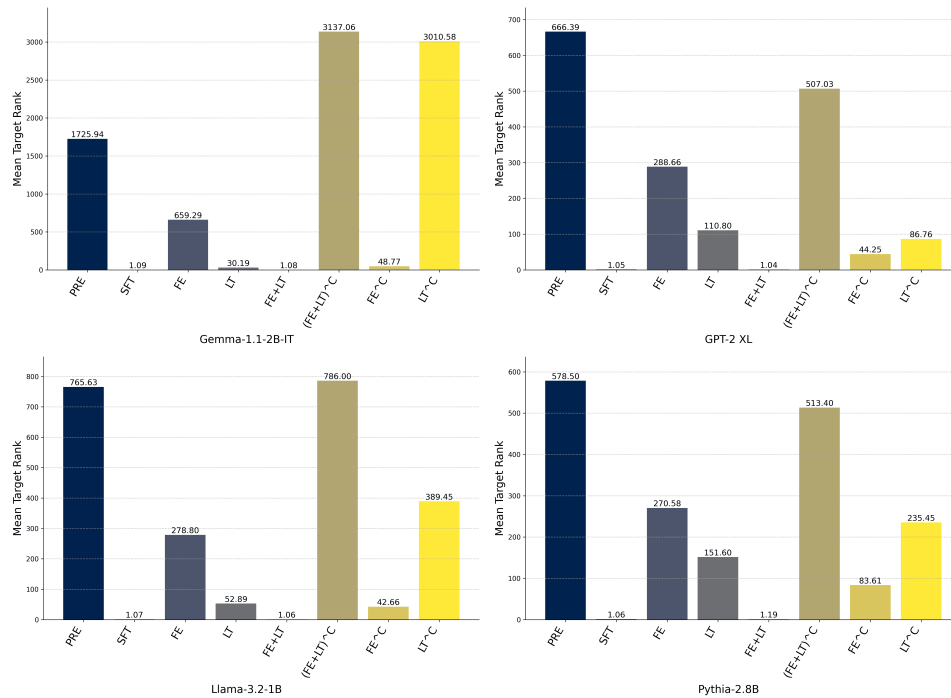


Figure 10: Mean target-token rank for Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We additionally evaluate alternative values of k on the Fake Movies, Real Actors dataset. The qualitative pattern is stable across k , although the individual “enrichment” and “recall” pathways are weaker for lower k values.

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

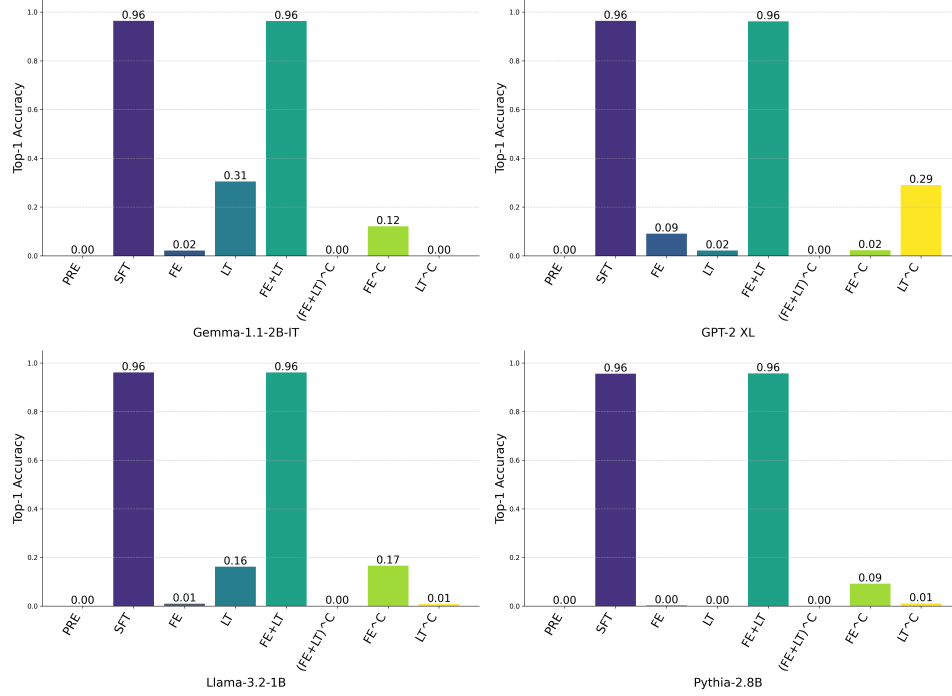


Figure 11: Top-1 accuracy for Sentence 1

C.2.2 TOP-10 RESULTS

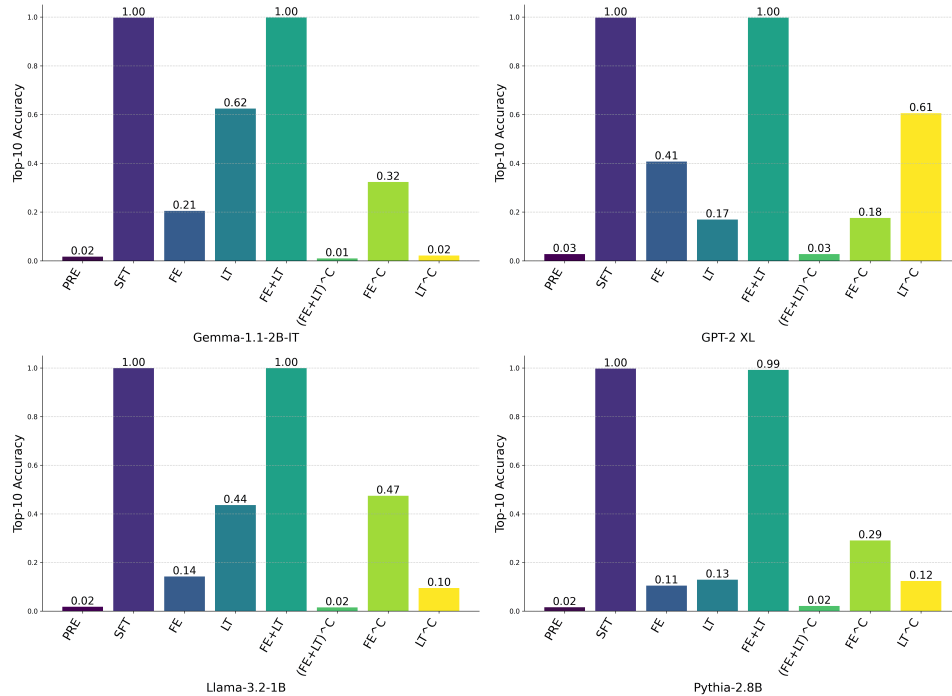


Figure 12: Top-10 accuracy for Sentence 1

C.2.3 TOP-100 RESULTS

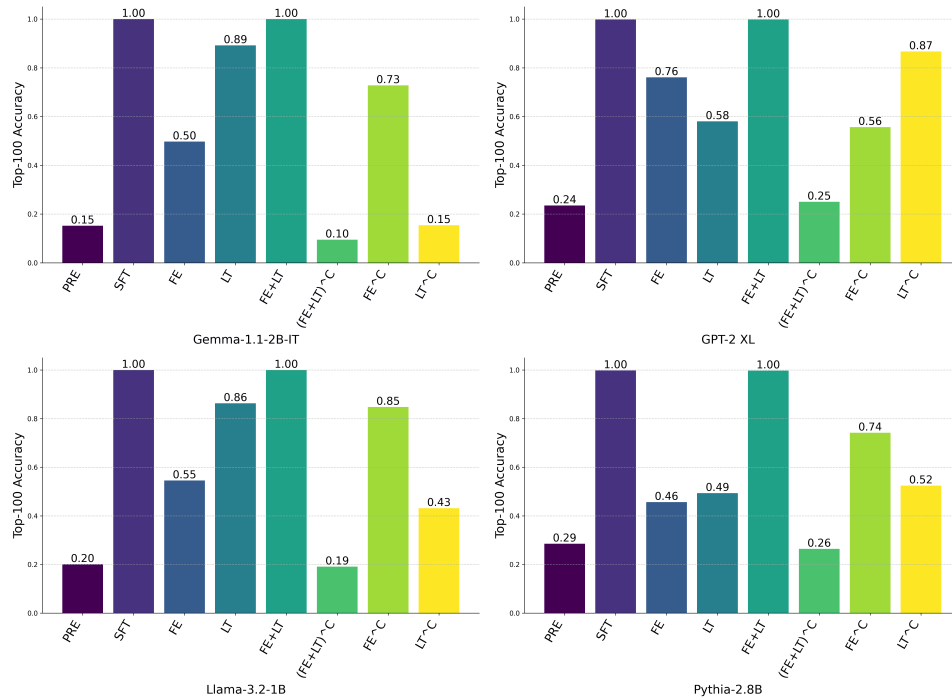


Figure 13: Top-10 accuracy for Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

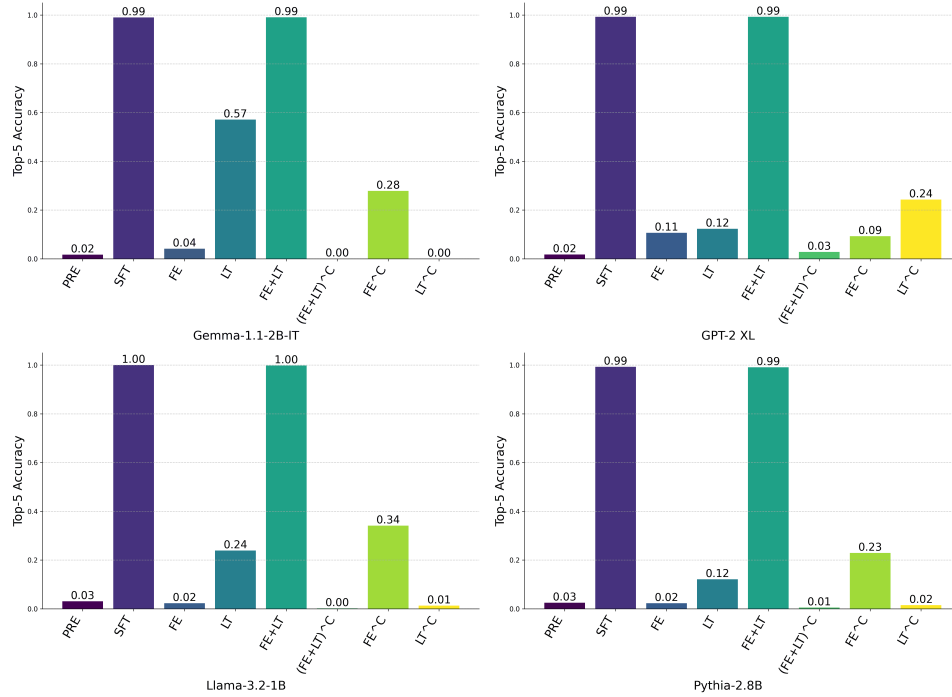


Figure 14: Top-5 accuracy for Sentence 1

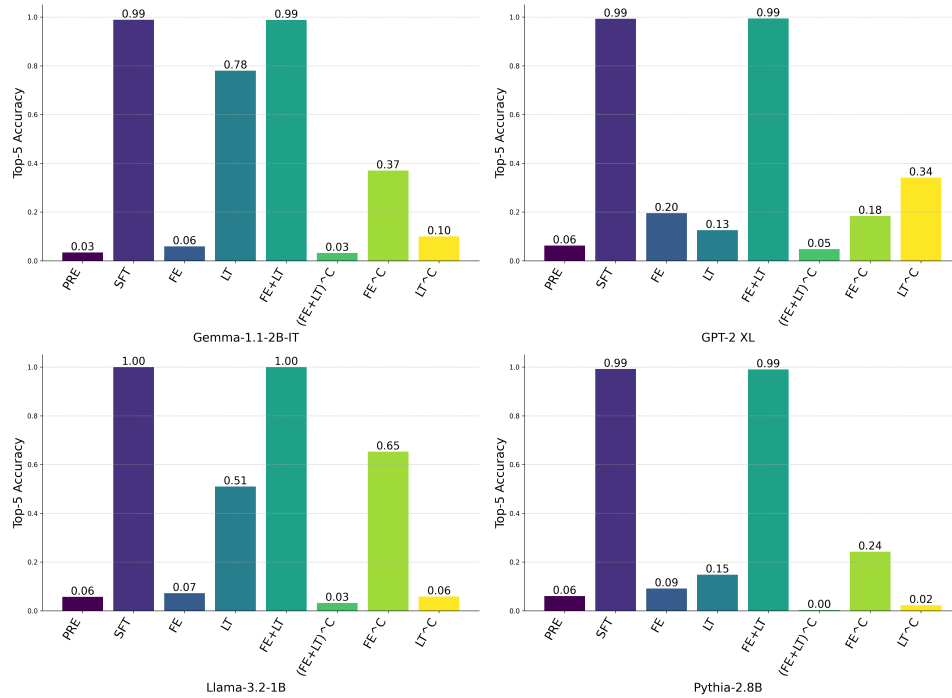


Figure 15: Top-5 accuracy for Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

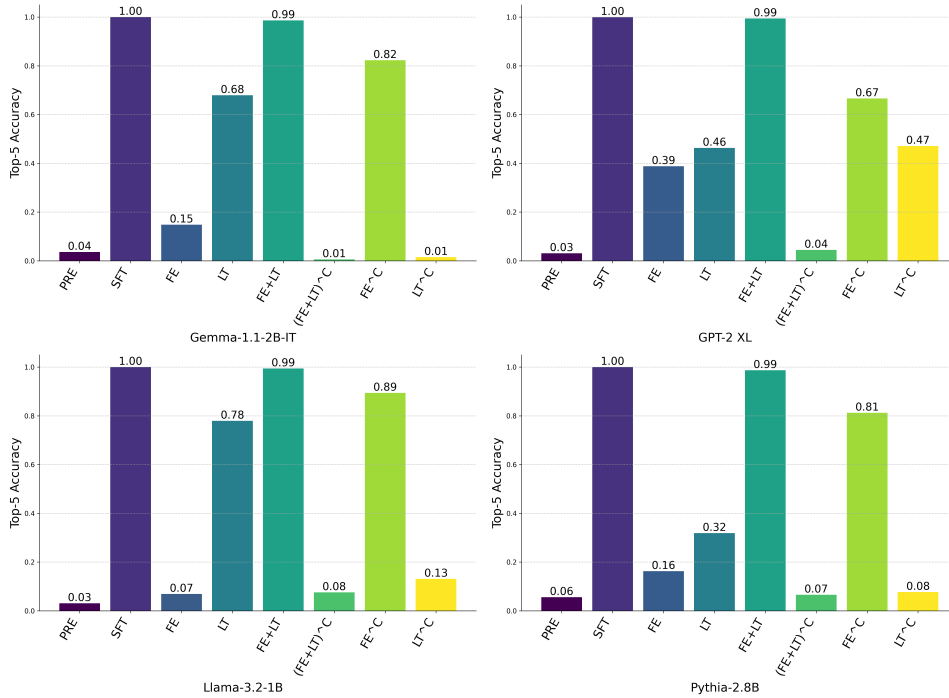


Figure 16: Top-5 accuracy for Sentence 1

C.5 MOVIE TITLE RESULTS

These experiments evaluate dynamic weight grafting when the movie title is included in the test sentence. The movie title alone is insufficient to recover the correct entity, but combining the movie title with the last token improves performance for all models. Combining the movie title with the first entity is model-dependent: relative to the first entity alone, adding the movie title improves GPT-2 XL, produces minimal change for Llama 3 and Pythia, and reduces performance for Gemma. The evaluated sentence is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

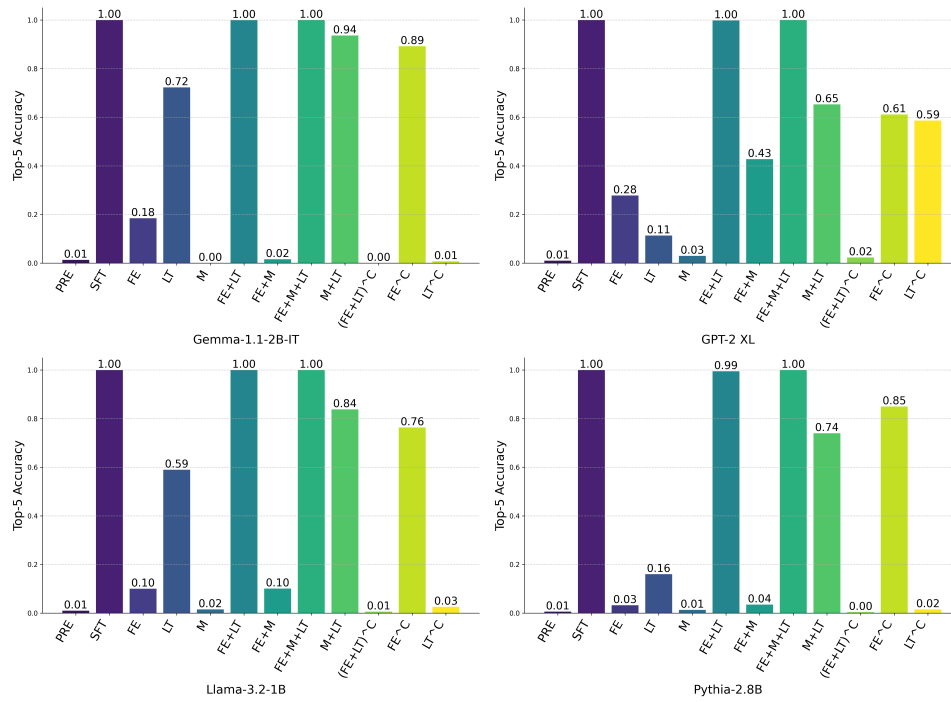


Figure 17: Top-5 accuracy for Sentence 3. “M” denotes the movie-title position.

C.6 UNEMBEDDING MATRIX RESULTS

These experiments use finetuned unembeddings. Although top-k accuracy changes in some configurations, especially for single-token positions, the qualitative pattern matches the results obtained with pretrained unembeddings.

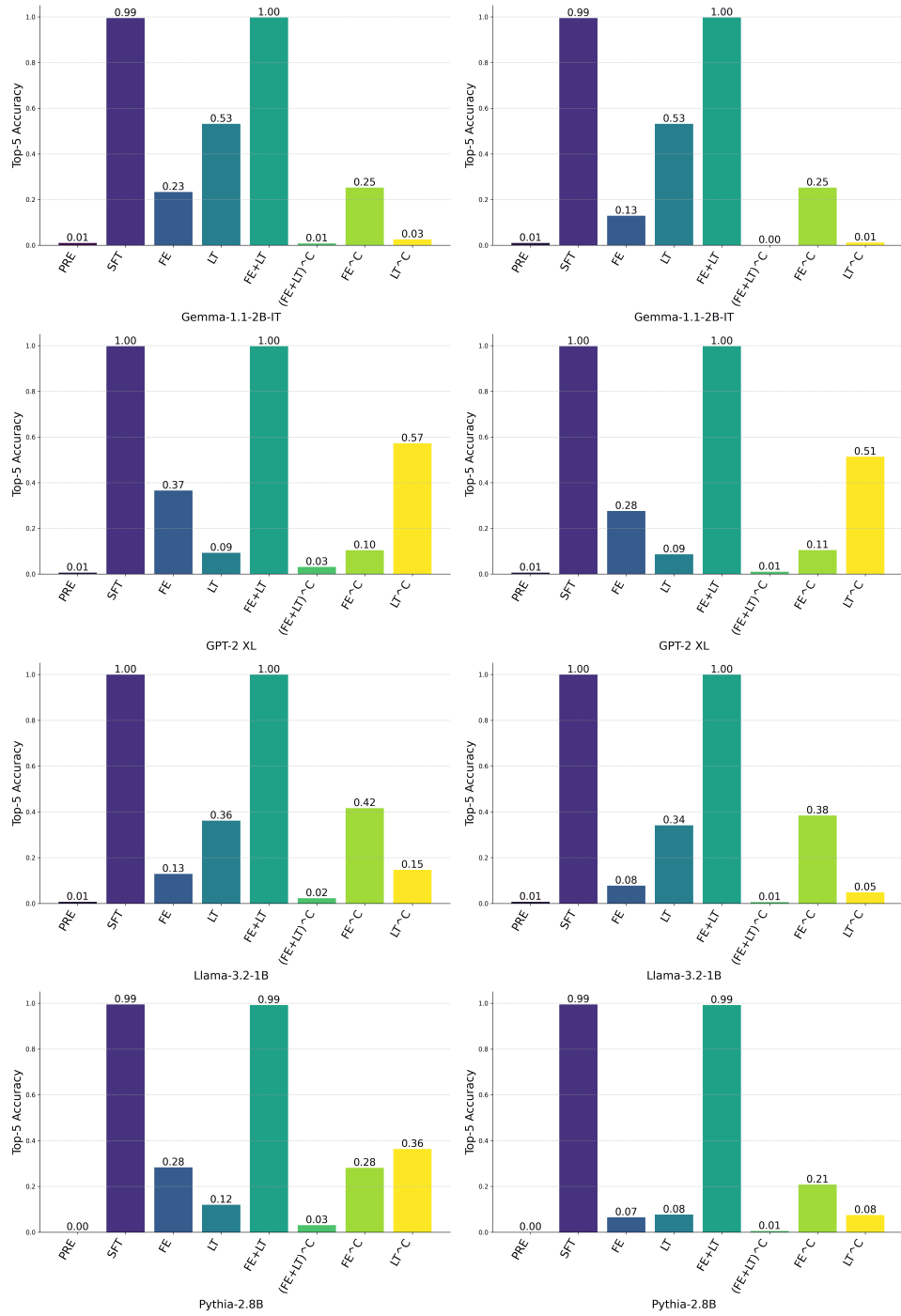


Figure 18: Top-5 accuracy for Sentence 1. Finetuned unembeddings are shown on the left and pretrained unembeddings on the right. Both unembedding choices produce similar patterns, with higher first-entity-only top-5 accuracy under finetuned unembeddings.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning condition, which uses a lower learning rate, 0 weight decay, and supplemental openwebtext and IMDB training data. The result pattern is consistent with the main experiments, but the individual “extraction” and “recall” pathways are weaker.

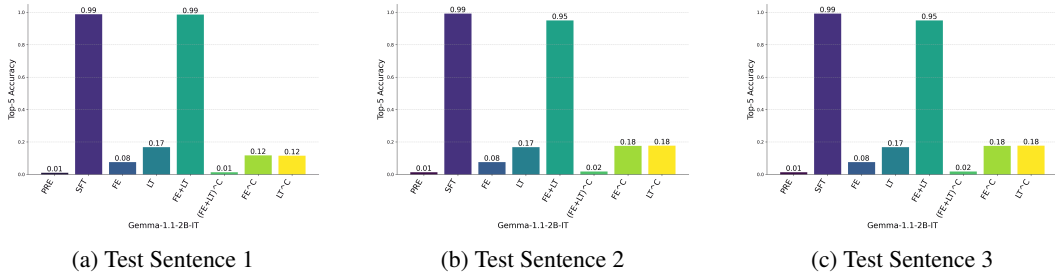


Figure 19: Top-5 accuracy for Gemma under lower-learning-rate finetuning with 0 weight decay and supplemental openwebtext and IMDB training data.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline component-grafting results for grafts from SFT models to pretrained models.

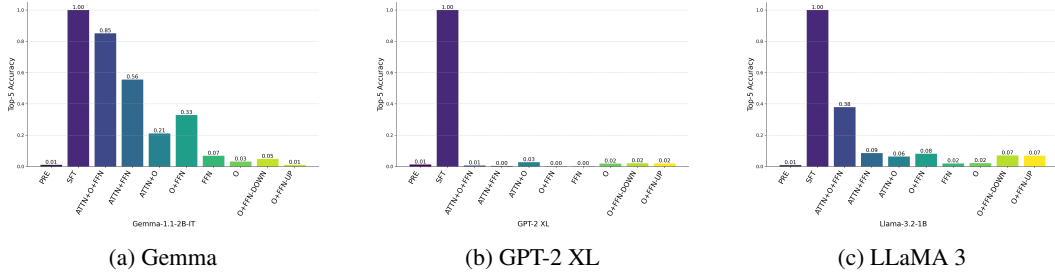


Figure 20: Top-5 accuracy for component-grafting baselines on Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

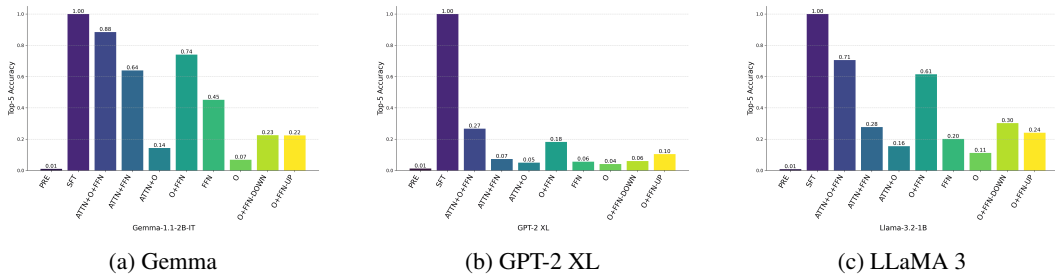


Figure 21: Top-5 accuracy for reversal-curse component-grafting results on Sentence 1

1566 C.10 TOKEN PROBABILITIES

1567
1568 We report five randomly sampled token-probability outputs that list the top 10 tokens for representative
1569 experimental examples. Although evaluation uses top-5 accuracy, the top-10 lists provide additional
1570 context about the next-token distributions.
1571

1572 C.10.1 LLAMA3: FE^C

1573
1574 Example 1:
1575 Target: Carolyn: 0.002
1576 J: 0.113
1577 L: 0.041
1578 Ch: 0.040
1579 Julie: 0.034
1580 Nicole: 0.024
1581 Nick: 0.023
1582 Michael: 0.023
1583 Jon: 0.023
1584 Jamie: 0.021
1585 Kelly: 0.017

1586 -----
1587 Example 2:
1588 Target: Lindsay: 0.024
1589 Michael: 0.093
1590 Ashley: 0.061
1591 Jennifer: 0.038
1592 Kim: 0.032
1593 Alexander: 0.025
1594 Lindsay: 0.024
1595 Milo: 0.023
1596 L: 0.021
1597 Jason: 0.018
1598 Lisa: 0.018

1599 -----
1600 Example 3:
1601 Target: Gwen: 0.015
1602 Michael: 0.030
1603 Paul: 0.028
1604 Julie: 0.024
1605 Elizabeth: 0.020
1606 Mary: 0.020
1607 S: 0.019
1608 E: 0.018
1609 J: 0.017
1610 L: 0.017
1611 A: 0.017

1612 -----
1613 Example 4:
1614 Target: Dominic: 0.006
1615 Is: 0.039
1616 Sarah: 0.031
1617 Jason: 0.026
1618 D: 0.026
1619 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1620 Mark: 0.019

1621

1622 -----

1623

1624 Example 5:

1625 Target: Ch: 0.045

1626 Ch: 0.045

1627 David: 0.024

1628 John: 0.022

1629 Peter: 0.018

1630 L: 0.018

1631 Mark: 0.018

1632 Michael: 0.017

1633 Richard: 0.016

1634 Ben: 0.012

1635 James: 0.012

1636

1637 -----

1638

1637 C.10.2 GEMMA: LT

1638

1639 Example 1:

1640 Target: Elizabeth: 0.040

1641 Julie: 0.495

1642 John: 0.130

1643 Stephen: 0.076

1644 Elizabeth: 0.040

1645 Hilary: 0.021

1646 Laurel: 0.018

1647 Tyne: 0.017

1648 Marian: 0.014

1649 Victoria: 0.013

1650 Juliet: 0.008

1651

1652 -----

1651 Example 2:

1652 Target: Uta: 0.008

1653 John: 0.901

1654 Joan: 0.022

1655 Sally: 0.011

1656 Chelsea: 0.009

1657 Uta: 0.008

1658 Tyne: 0.003

1659 Gina: 0.002

1660 Elle: 0.001

1661 Lisa: 0.001

1662 Rosemary: 0.001

1663

1664 -----

1664 Example 3:

1665 Target: Jennifer: 0.706

1666 Jennifer: 0.706

1667 John: 0.094

1668 Il: 0.028

1669 Me: 0.020

1670 Stephen: 0.015

1671 Carol: 0.012

1672 S: 0.008

1673 David: 0.008

1674 Elle: 0.007

1675 T: 0.006

```
1674 -----
1675
1676 Example 4:
1677 Target: Marcia: 0.017
1678 John: 0.152
1679 Gwen: 0.136
1680 Susan: 0.050
1681 Tim: 0.045
1682 Ch: 0.041
1683 Celeste: 0.037
1684 Tara: 0.030
1685 T: 0.028
1686 Elle: 0.028
1687 Brittany: 0.026
1688 -----
```

```
1689 Example 5:
1690 Target: Heather: 0.035
1691 Ly: 0.916
1692 Heather: 0.035
1693 Hilary: 0.011
1694 Stephen: 0.007
1695 Julie: 0.002
1696 Rosemary: 0.002
1697 Ch: 0.002
1698 Wanda: 0.001
1699 Chelsea: 0.001
1700 Tem: 0.001
1701 -----
```

1702 C.10.3 GPT-2 XL: (FE+LT)^C

```
1703 Example 1:
1704 Target: Fre: 0.000
1705 her: 0.037
1706 John: 0.018
1707 Jason: 0.017
1708 Chris: 0.016
1709 Adam: 0.015
1710 Zach: 0.014
1711 Josh: 0.013
1712 the: 0.013
1713 Michael: 0.013
1714 Ben: 0.012
1715 -----
```

```
1716 Example 2:
1717 Target: A: 0.002
1718 her: 0.043
1719 the: 0.017
1720 John: 0.016
1721 Tom: 0.016
1722 Peter: 0.012
1723 fellow: 0.012
1724 Michael: 0.011
1725 David: 0.011
1726 James: 0.011
1727 Jack: 0.010
1728 -----
```

```
1728 Example 3:
1729 Target: Matthew: 0.003
1730 his: 0.057
1731 the: 0.017
1732 Jennifer: 0.014
1733 fellow: 0.012
1734 Chris: 0.011
1735 Michael: 0.011
1736 Jason: 0.009
1737 John: 0.008
1738 James: 0.008
1739 Jessica: 0.007
```

```
-----
1739
1740 Example 4:
1741 Target: Cher: 0.000
1742 her: 0.042
1743 Tom: 0.026
1744 Robert: 0.019
1745 the: 0.018
1746 Matt: 0.018
1747 Michael: 0.016
1748 Brad: 0.016
1749 Chris: 0.014
1750 Johnny: 0.014
1751 John: 0.014
```

```
-----
1751
1752 Example 5:
1753 Target: Timothy: 0.001
1754 his: 0.056
1755 the: 0.025
1756 Michael: 0.013
1757 fellow: 0.013
1758 Robert: 0.013
1759 John: 0.012
1760 James: 0.010
1761 Tom: 0.010
1762 Mark: 0.008
1763 Peter: 0.008
```

```
-----
```

1766 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1768 The following representative Gemma result, with grafting at the first-entity token, illustrates a
1769 distribution that assigns high probability to common tokens while still ranking the target token in the
1770 top 5.

```
1771
1772 Target: Robert: 0.016
1773 a: 0.365
1774 an: 0.151
1775 the: 0.091
1776 Robert: 0.016
1777 two: 0.009
1778 his: 0.008
1779 other: 0.006
1780 John: 0.005
1781 Morgan: 0.005
1782 my: 0.005
```